3. Autorisation : lecture et écriture d'une ressource

Quatre niveaux d'accès :

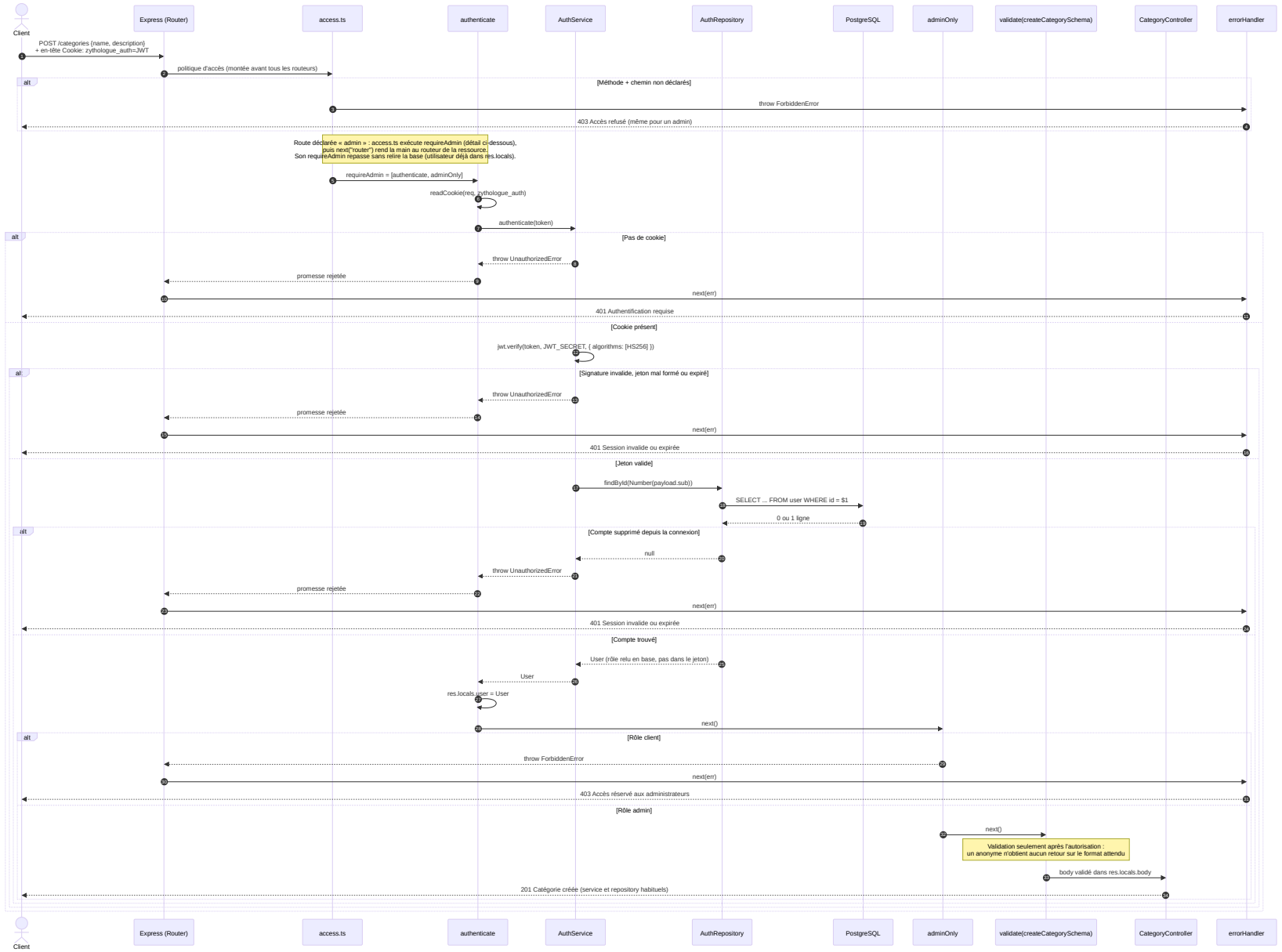
Niveau	Qui	Exemples	Vérifié par
Public	tout le monde	tous les `GET` du catalogue, `GET /beers/:id/reviews`	`access.ts` (déclaration seule)
Connecté	tout compte avec un jeton valide	`POST /beers/:id/reviews`, `GET /auth/me`	`access.ts` puis la route : middleware `authenticate`
Admin	rôle `admin`	écriture du catalogue, `GET /beer-logs`, `GET /stats`	`access.ts` puis la route : `requireAdmin` = `[authenticate, adminOnly]`
Propriétaire	l'auteur de la donnée (ou un admin pour la suppression)	`PATCH` / `DELETE /beers/:id/reviews/:reviewId`	`BeerReviewService` (règle métier)

Avant ces contrôles, la politique d'accès `access.ts` vérifie que le couple méthode + chemin est déclaré, et applique le niveau prévu. Toute route non déclarée est refusée (**403 « Accès refusé »**, même pour un admin). Les middlewares des fichiers de routes restent en place en double protection.

Codes d'erreur : **401** signifie « je ne sais pas qui tu es » (pas de jeton, jeton invalide ou expiré). **403** signifie « je sais qui tu es, mais tu n'as pas le droit ».

3a. Le middleware `authenticate` + `requireAdmin` (écriture dans le catalogue)

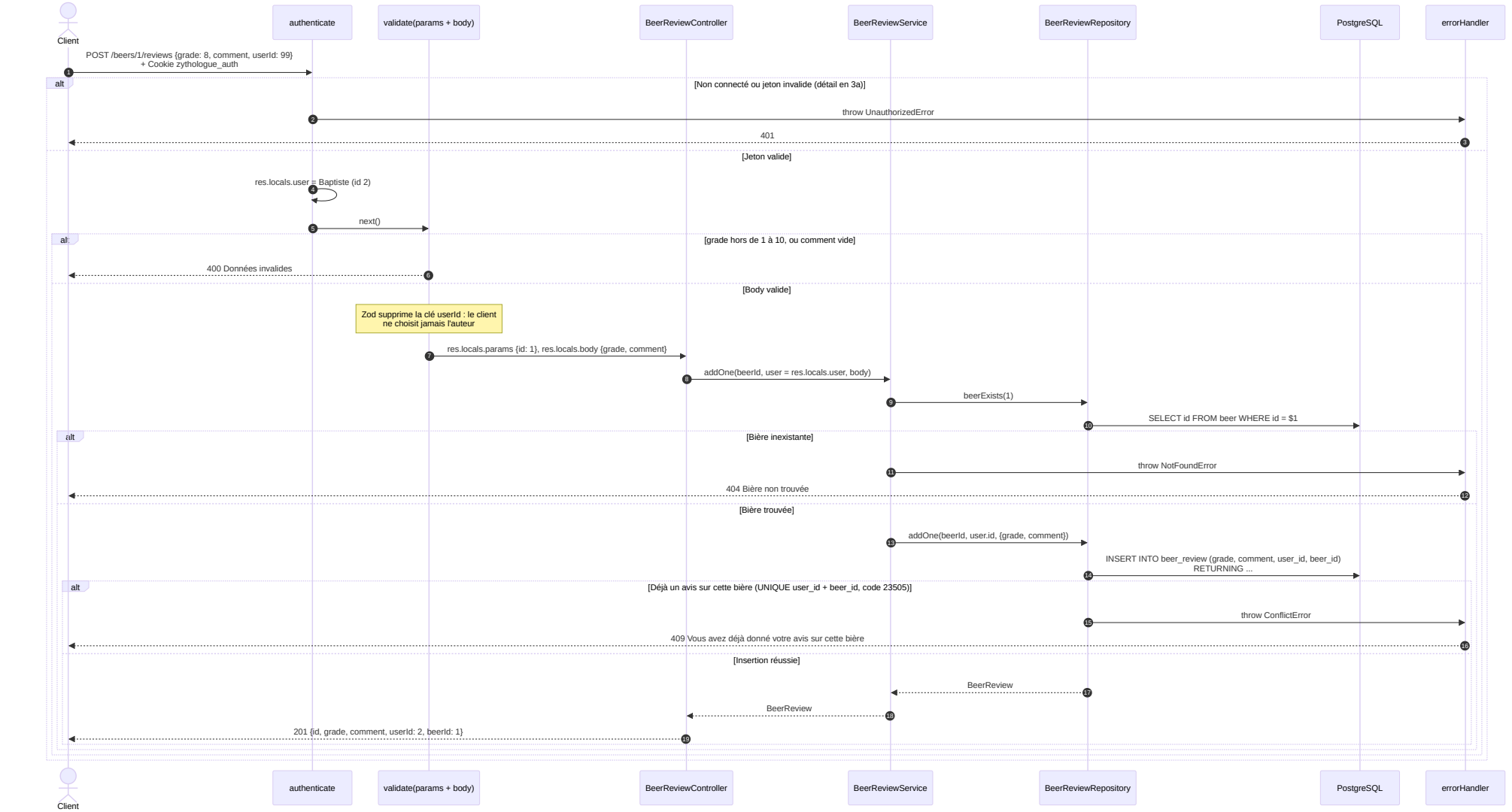
Exemple : `POST /api/v1/categories`. Le même enchaînement protège tous les `POST`, `PATCH` et `DELETE` du catalogue.


Client
Express (Router)
access.ts
authenticate
AuthService
AuthRepository
PostgreSQL
adminOnly
validate(createCategorySchema)
CategoryController
errorHandler
POST /categories (name, description)
+ en-tête Cookie: zythologue_auth=JWT
politique d'accès (montée avant tous les routeurs)
alt
[Méthode + chemin non déclarés]
throw ForbiddenError
403 Accès refusé (même pour un admin)
Route déclarée « admin » : access.ts exécute requireAdmin (détail ci-dessous),
puis next("router") rend la main au routeur de la ressource.
Son requireAdmin repasse sans relire la base (utilisateur déjà dans res.locals).
requireAdmin = [authenticate, adminOnly]
readCookie(req, zythologue_auth)
authenticate(token)
alt
[Pas de cookie]
throw UnauthorizedError
promesse rejetée
next(err)
401 Authentification requise
[Cookie présent]
jwt.verify(token, JWT_SECRET, { algorithms: [HS256] })
alt
[Signature invalide, jeton mal formé ou expiré]
throw UnauthorizedError
promesse rejetée
next(err)
401 Session invalide ou expirée
[Jeton valide]
findById(Number(payload.sub))
SELECT ... FROM user WHERE id = $1
0 ou 1 ligne
alt
[Compte supprimé depuis la connexion]
null
throw UnauthorizedError
promesse rejetée
next(err)
401 Session invalide ou expirée
[Compte trouvé]
User (rôle relu en base, pas dans le jeton)
User
res.locals.user = User
next()
alt
[Rôle client]
throw ForbiddenError
next(err)
403 Accès réservé aux administrateurs
[Rôle admin]
next()
Validation seulement après l'autorisation :
un anonyme n'obtient aucun retour sur le format attendu
body validé dans res.locals.body
201 Catégorie créée (service et repository habituels)

3b. Création d'un avis : l'écriture porte l'utilisateur du jeton

Exemple : Baptiste (client, id 2) note la bière 1 et tente de se faire passer pour l'utilisateur 99.

> Pour la lisibilité, les erreurs sont dessinées directement vers `errorHandler`. Le chemin réel est le même qu'en 3a : throw, puis promesse rejetée dans le controller, puis `next(err)` par Express.
>
> `access.ts` n'est pas dessiné en 3b et 3c : ces routes y sont déclarées « connecté », il exécute donc `authenticate` en premier (comme en 3a). Le `authenticate` de la route trouve ensuite l'utilisateur déjà dans `res.locals` et ne relit pas la base.


Client
authenticate
validate(params + body)
BeerReviewController
BeerReviewService
BeerReviewRepository
PostgreSQL
errorHandler
POST /beers/1/reviews {grade: 8, comment, userId: 99}
+ Cookie zythologue_auth
alt
[Non connecté ou jeton invalide (détail en 3a)]
throw UnauthorizedError
401
[Jeton valide]
res.locals.user = Baptiste (id 2)
next()
alt
[grade hors de 1 à 10, ou comment vide]
400 Données invalides
[Body valide]
Zod supprime la clé userId : le client
ne choisit jamais l'auteur
res.locals.params {id: 1}, res.locals.body {grade, comment}
addOne(beerId, user = res.locals.user, body)
beerExists(1)
SELECT id FROM beer WHERE id = $1
alt
[Bière inexistante]
throw NotFoundError
404 Bière non trouvée
[Bière trouvée]
addOne(beerId, user.id, {grade, comment})
INSERT INTO beer_review (grade, comment, user_id, beer_id)
RETURNING ...
alt
[Déjà un avis sur cette bière (UNIQUE user_id + beer_id, code 23505)]
throw ConflictError
409 Vous avez déjà donné votre avis sur cette bière
[Insertion réussie]
BeerReview
BeerReview
201 {id, grade, comment, userId: 2, beerId: 1}

3c. Modification et suppression d'un avis : la règle du propriétaire

La route ne vérifie que l'identité (`authenticate`). Savoir si l'avis t'appartient demande de **le lire en base** : ce contrôle est donc une règle métier, écrite dans le service.

